

Github Made Easy : Basics of Github

Table of Contents

Topic	Page
Account Setup	01
Create a Repository (Project Folder)	02
Add Files to Your Repository	03
Understand: README.md, .gitignore, and LICENSE	05
Free Website Hosting (Enable GitHub Pages)	07
Preparing & Uploading HTML/CSS/JS to GitHub Properly	09
Meta-Prompting	11
Github Desktop - Account and Repo Setup	12
Github Desktop - Basic Repo Commits	17

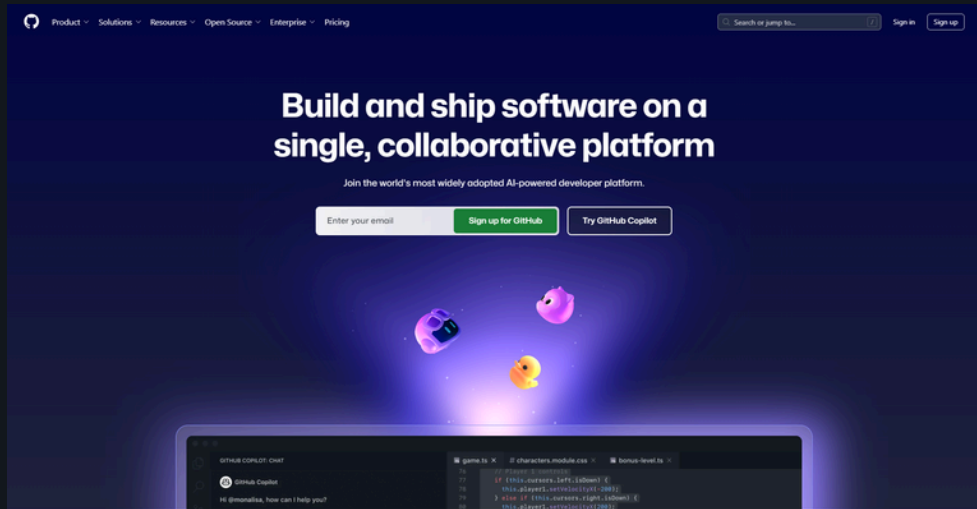
Click the page number to navigate directly to that page.

This PDF/guide was created by [Runarok](#).

To find the latest version of the PDF guide, visit : [Runarok Github Manual](#)

Account Setup

Navigate to GitHub : Go to [Github](#)



If You're New:

Click Sign Up

Fill in: Email , Password , Username

Choose Free plan

-(Pro not needed for public site hosting)

Complete CAPTCHA & setup

A screenshot of the GitHub account creation process. On the left, a dark sidebar with the text 'Create your free account' and 'Explore GitHub's core features for individuals and organizations.' Below this is a small image of three GitHub Octocat avatars. The main area is a white form titled 'Sign up to GitHub'. It contains fields for 'Email', 'Password', and 'Username'. Below the 'Username' field is a note: 'Username may only contain alphanumeric characters or single hyphens, and cannot begin or end with a hyphen.' There is a dropdown for 'Your Country/Region' with 'India' selected. Below this is a checkbox for 'Email preferences' with the text 'Receive occasional product updates and announcements'. At the bottom is a 'Continue >' button. A small disclaimer at the very bottom states: 'By creating an account, you agree to the [Terms of Service](#). For more information about GitHub's privacy practices, see the [GitHub Privacy Statement](#). We'll occasionally send you account related emails.'

If You Already Have an Account:

Click Sign In (top-right)

Enter your username/email and password

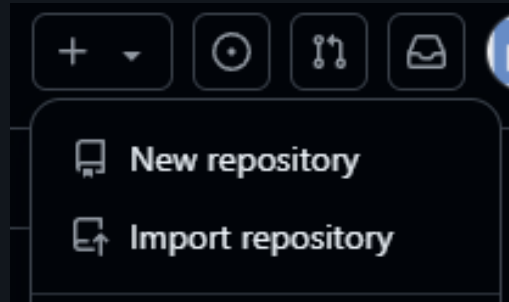
A screenshot of the GitHub sign-in page. It features the GitHub logo at the top. Below it is the text 'Sign in to GitHub'. There are two input fields: 'Username or email address' and 'Password'. To the right of the password field is a link that says 'Forgot password?'. Below the input fields is a large green button labeled 'Sign in'. At the bottom, there is a section for 'Sign in with a passkey' and a link for 'New to GitHub? Create an account'.

[→ Back to Table of Contents](#)

Create a Repository (Project Folder)

GitHub gives you two options when creating a new project:

Click the "+" icon (top-right of any GitHub page)



New Repository (Start from Scratch)



Create a new repository

A repository contains all project files, including the revision history. Already have a project repository elsewhere? [Import a repository.](#)

Required fields are marked with an asterisk (*).

Owner * Repository name *

Runarok /

Great repository names are short and memorable. Need inspiration? How about [psychic-guide](#) ?

Description (optional)

☒ **Public**
Anyone on the internet can see this repository. You choose who can commit.

☐ **Private**
You choose who can see and commit to this repository.

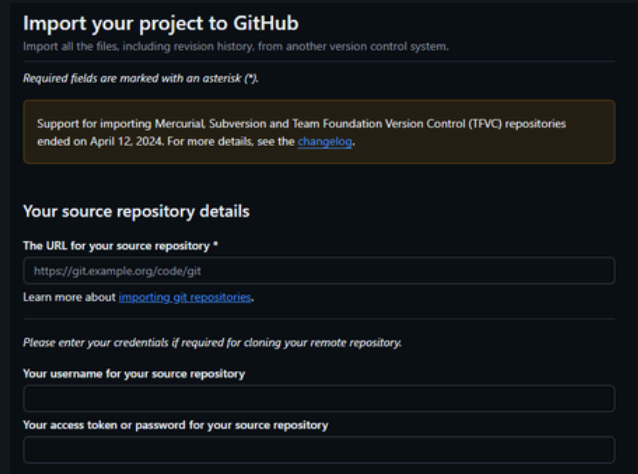
Use If:

- Starting a new project from scratch
- Uploading files manually (no importing)

Steps:

- Enter repository name
 - → e.g., my-website, project-1
- (Optional) Add a short description
- Choose visibility → Select Public (required for Free Pages hosting)
- Optional initializations
 - README.md , License , .gitignore
- Click Create repository

Import Repository (From another platform)



Import your project to GitHub

Import all the files, including revision history, from another version control system.

Required fields are marked with an asterisk (*).

Support for importing Mercurial, Subversion and Team Foundation Version Control (TFVC) repositories ended on April 12, 2024. For more details, see the [changelog](#).

Your source repository details

The URL for your source repository *

Learn more about [importing git repositories](#).

Please enter your credentials if required for cloning your remote repository.

Your username for your source repository

Your access token or password for your source repository

Use this if:

- You want to copy code from a GitLab, Bitbucket, or other repository to GitHub

Steps:

- Paste the URL of the existing repository
- Choose a new name for your GitHub repo
- Choose Public or Private
- Click Begin Import

GitHub will copy the files and commit history from that repo.

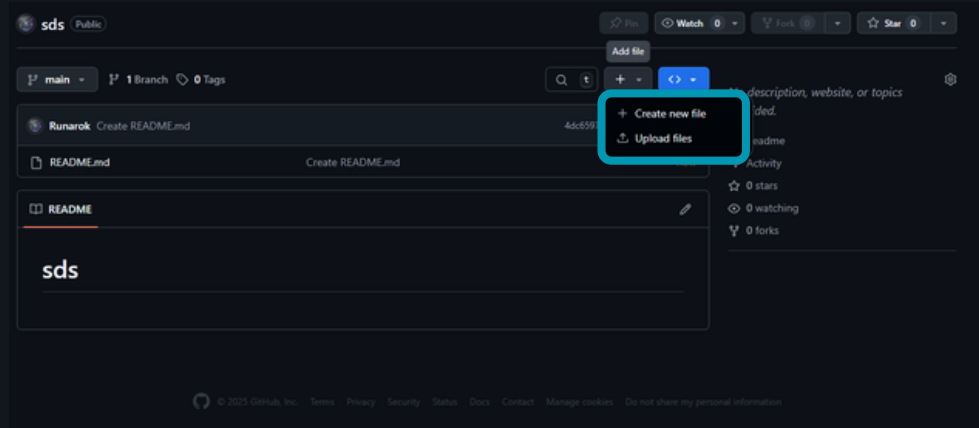
[→ Back to Table of Contents](#)

Add Files to Your Repository

Firstly, Was the Repo Initialized?

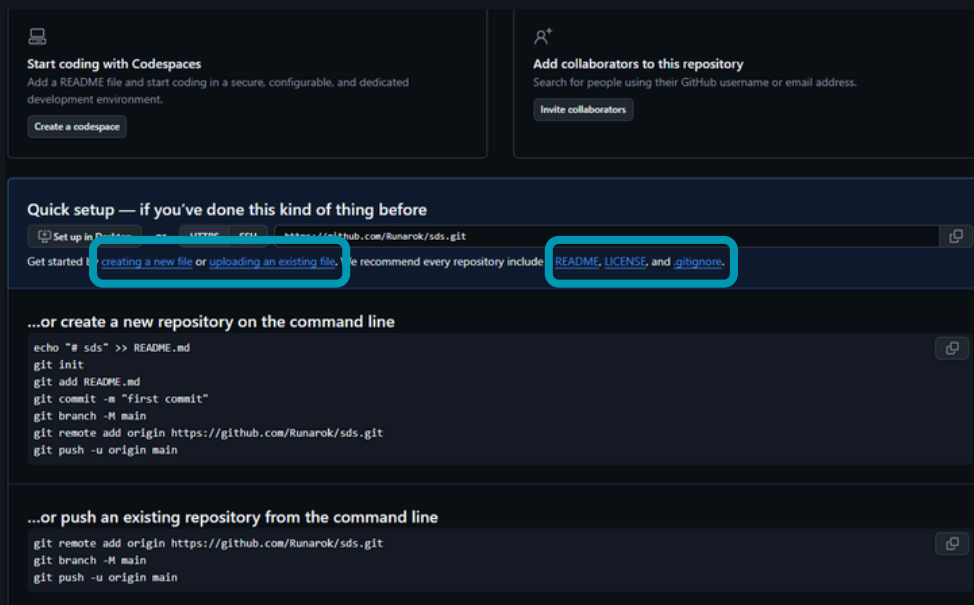
If you selected **README / License / .gitignore** during creation:

→ You'll see a file list right away (repo is initialized)



If you skipped all initialization:

→ GitHub shows "Quick setup" page (repo is empty)



→ [Back to Table of Contents](#)

Upload Files (For bulk/manual upload)

Use If:

- You already have HTML/CSS/image files on your computer
- You want to drag and drop/upload multiple files

Where:

- In the repo → Click "Add file" → Upload files

Steps:

1. Drag & drop files (e.g., index.html, about.html, style.css)
2. Add Commit message and Description(Optional)
3. Scroll down → Add commit message

Can be repeated anytime to update or replace files.

Create New File (For writing online)

Use If:

- You want to create a small file manually (e.g., index.html)
- You're just editing or adding a page/content directly in browser

Where:

- Click "Add file" → Create new file

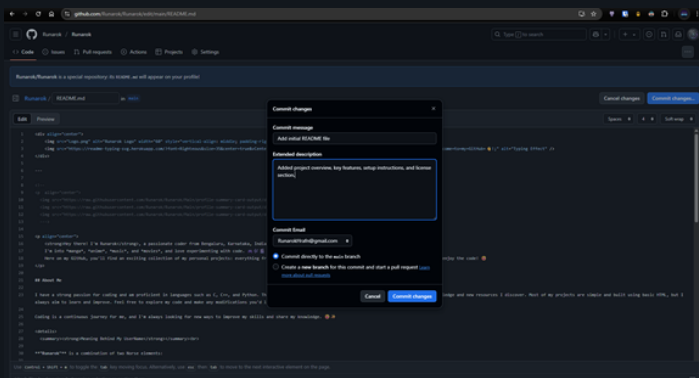
Steps:

1. Name the file (e.g., index.html)
2. Add your content (HTML, Markdown, etc.)
3. Scroll up → click Commit changes
4. Add a commit message (optional description)
5. Click Commit changes to save it

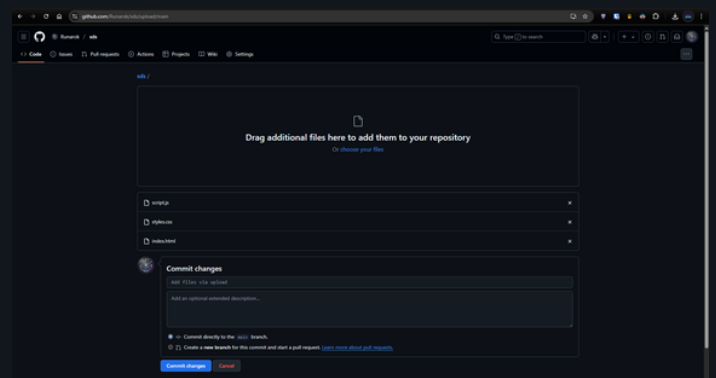
You can create as many files as you want one after another this way.

Examples:

Create New File



Upload Files



[→ Back to Table of Contents](#)

Understand: README.md, .gitignore, and LICENSE

README.md — Project Overview (Markdown Format)

Purpose:

- Shown on your GitHub repo page, not on the hosted site
- Written in GFM (GitHub Flavored Markdown)

Can Include:

- #, ##, ### → Headings
- [text](url) → Links
- ```code``` → Code blocks
- Lists, tables, bold/italic, etc.

Visibility:

- Always shown at github.com/username/repo-name
- If repo = username, it's shown on:
 - github.com/username

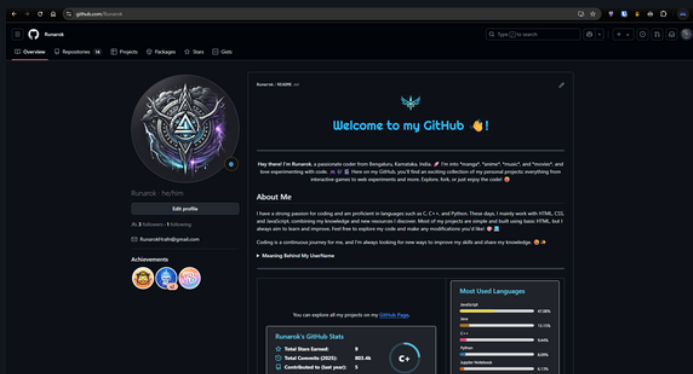
Use For:

- Explaining your project or website
- How to use/run/view
- Tool credits (e.g., "Made with ChatGPT")
- Any text you want in rich format via Markdown

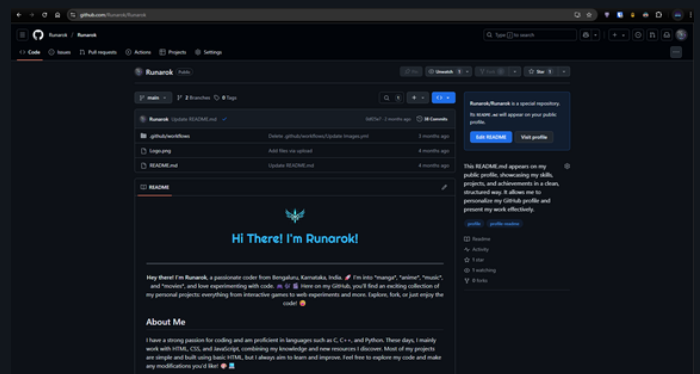
Tip: New to Markdown? Check out this [Markdown guide](#) for a quick overview of formatting options.

Examples:

README shown at [My Profile URL](#)



README shown in [My Profile](#) repo



[→ Back to Table of Contents](#)

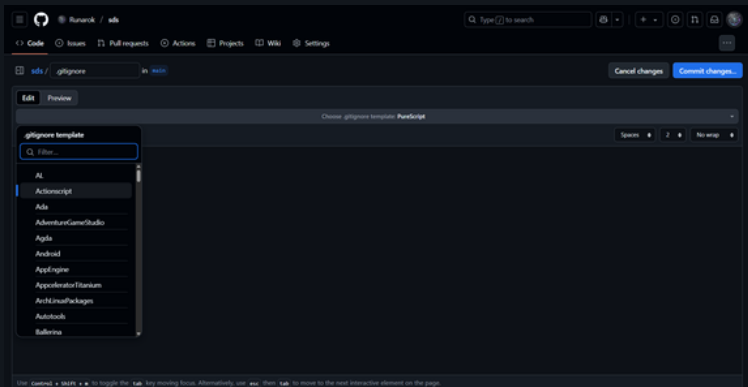
.gitignore — Ignore Unwanted Files

Purpose:

- Tells Git what not to track (e.g., system/dev files)

Only useful if:

- You're using:
 - Git CLI (terminal)
 - GitHub Desktop
- Not useful if:
 - You only upload files via GitHub website (manual uploads)



During file creation on GitHub, you can:

- Enter the file name (gitignore) and use templates if available (for certain file types)
- Or Write your content directly in the editor
- Commit changes with a message to save the file

Tip: New to .gitignore? Check out [W3Schools](https://www.w3schools.com/gitignore/) and [freeCodeCamp](https://www.freecodecamp.org/gitignore/) for a clear understanding.

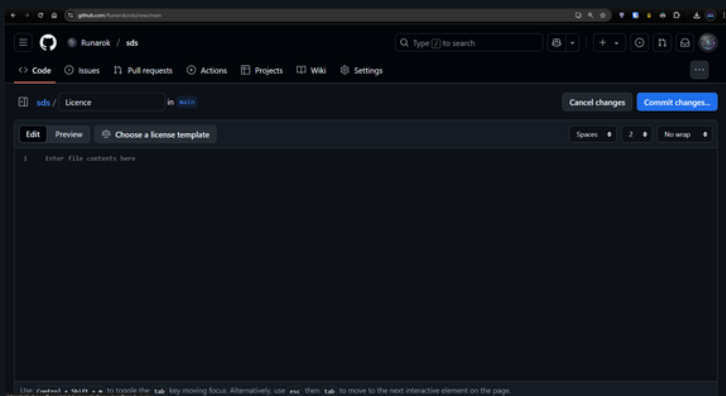
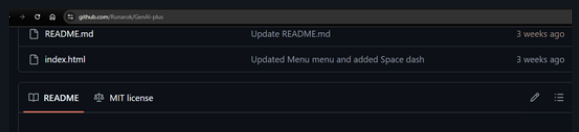
LICENSE — Open Source Permissions

Purpose:

- Legally declares how others may use your code/files
- Visible publicly in the repo - Required if you want to make your project truly "open source"

Where It Appears:

- GitHub shows it as a label/tag on the repo
- Also visible as a file (**LICENSE**) in the repo's file list



During file creation on GitHub, you can:

- Enter the file name (Licence) and use templates Or Write your content directly in the editor
- Commit changes with a message to save the file

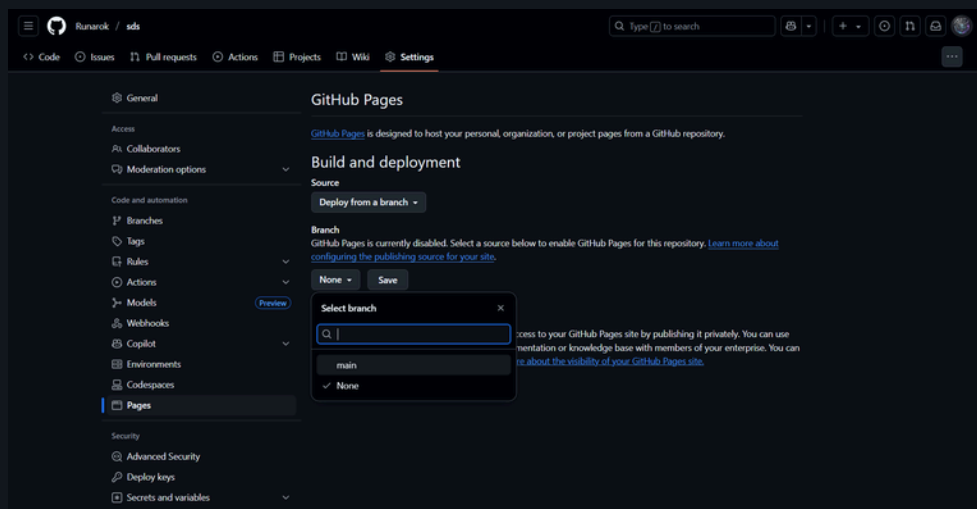
Tip: New to GitHub licensing? Visit [OSI](https://opensource.org/licenses/) for license details and choosealicense.com to pick the right one.

[→ Back to Table of Contents](#)

Free Website Hosting (Enable Github Pages)

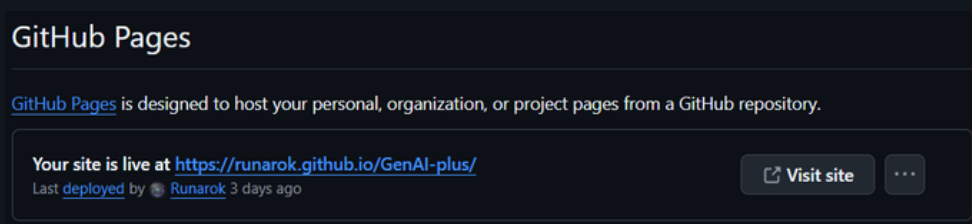
Steps to Host Any Repository Site:

- Go to your repository on GitHub
- Click Settings → Left sidebar → Pages
- Under Source, choose your Branch: main / master
- Folder: / (root) (or /docs if your files are inside a folder named docs) then Click Save



After You Click Save:

- GitHub automatically builds and hosts your site
- Wait for a while - A box will appear showing the live link:
 - Your site is live at: <https://username.github.io/repo-name>
 - Copy and share the link! It becomes active within 10-60 seconds.



Special Case: Personal Main Site (Main Portfolio Or Site)

- Repository name: username.github.io (Replace username with your actual GitHub username)
- Required page: [index.html](#) (This will serve as the homepage of your personal site)
- Site URL: <https://username.github.io/>
- Important notes:
 - You can only have one personal main site like this per GitHub account.
 - This site acts as your primary portfolio or personal homepage on GitHub Pages.

[→ Back to Table of Contents](#)

Project Sites (Multiple Repositories)

- **Repository name:** Any name (e.g., portfolio, blog, project-x)
- **Required page:** `index.html` (placed in the root or docs/ folder)
- **Site URL:** `https://username.github.io/project-name`
- **Important notes:**
 - Create as many project sites as you want this way.

File / Folder Structure	Example File	Link Example	Resulting URL
File in root folder	<code>about.html</code>	<code>About</code>	<code>https://username.github.io/repo-name/about.html</code>
File in root folder	<code>contact.html</code>	<code>Contact</code>	<code>https://username.github.io/repo-name/contact.html</code>
File inside folder	<code>pages/info.html</code>	<code>Info</code>	<code>https://username.github.io/repo-name/pages/info.html</code>
File inside nested folder	<code>blog/post1.html</code>	<code>Blog Post 1</code>	<code>https://username.github.io/repo-name/blog/post1.html</code>

Tip: [GitLinkSyncer](#) lets you input any GitHub URL and instantly get its raw, repo, and GitHub Pages versions—great for checking subpage links.

GitHub ↔ Pages Link Analyzer

Paste any GitHub, GitHub Pages, or Raw link:

GitHub Repository or File:

Copy Link

Open Link

GitHub Pages Site or Subpath:

Copy Link

Open Link

Raw File Content URL:

Copy Link

Open Link

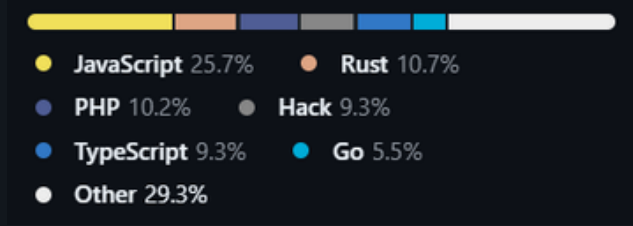
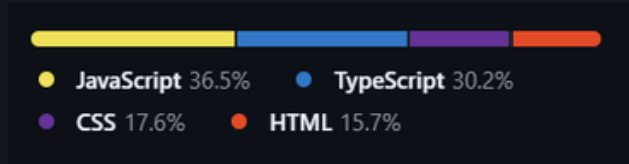
[→ Back to Table of Contents](#)

Preparing & Uploading HTML/CSS/JS to GitHub Properly

Step 1: Understand How GitHub Detects Languages

- On your repository page (top right section), GitHub automatically shows what languages are used and in what percentage.

◦ Example breakdown:



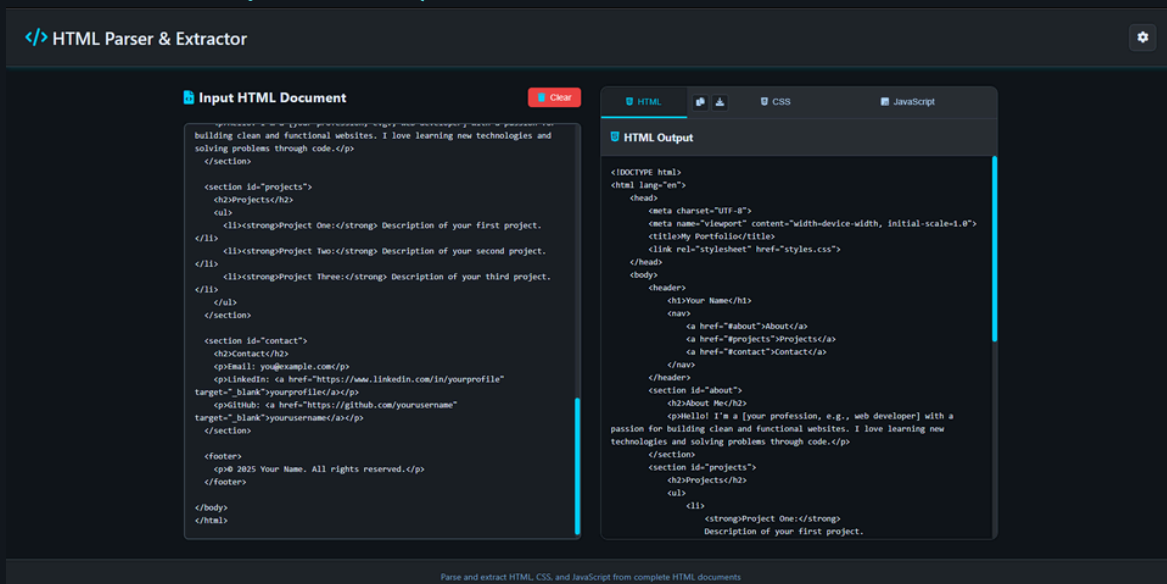
GitHub only detects languages based on the actual files present in your repo (e.g., .html, .css, .js).

Step 2: Know the Problem With Combined Files

- If your website code is in a single .html file (copied from somewhere with inline CSS and JS):
- GitHub will only count it as HTML
- CSS and JavaScript won't show up as separate languages
- Your repo will look incomplete or unstructured
- Not good for readability, reusability, or hosting dynamic sites

Step 3: Split Combined Code into HTML + CSS + JS

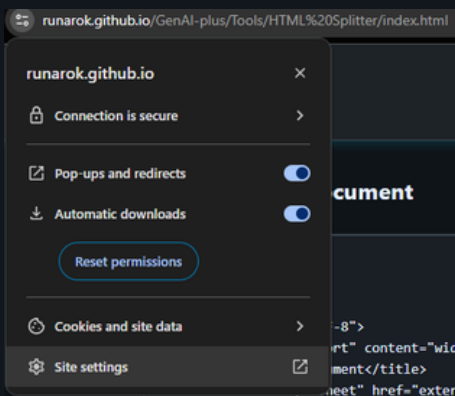
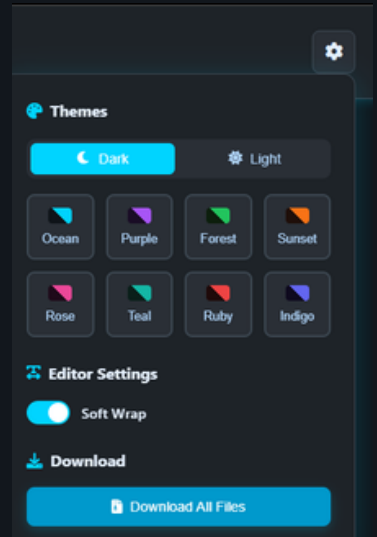
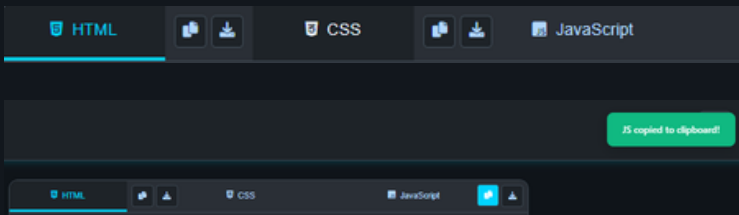
- Go to this tool: [HTML Splitter](#)
- Paste your entire HTML file (even if it includes CSS/JS) into the left side of the screen.
- The tool will automatically split the content into:
 - index.html, style.css, script.js



→ [Back to Table of Contents](#)

- You can hover over the right tabs (HTML / CSS / JS) to:
- Download each file individually
- OR click the Settings (top right) → Click "Download All"

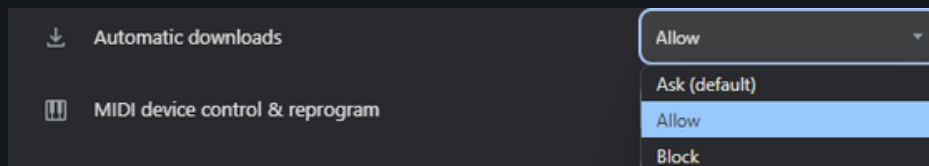
You can also manually copy and paste each part if preferred.



Note : Allow Multi-file Downloads (if blocked)

- If you get blocked when trying to download all files:
- Look at the top-left of your browser's address bar
- Click the  (lock icon) → Site settings
- Find "Automatic downloads" and set it to Allow

Now the multi-file download should work.



Step 4: Organize and Upload to GitHub

- Create or open your GitHub repository
- Click "Add file" → Upload files
- Upload:
 - index.html → Root folder
 - script.js → Either root or js/ folder
 - style.css → Either root or css/ folder

If using folders, make sure your HTML file links correctly:

`<link rel="stylesheet" href="css/style.css">` , `<script src="js/script.js"></script>`

As by default they are set to be in same root folder as the index.html

- Result:*
- GitHub now detects HTML, CSS, and JavaScript correctly
 - Your repo appears professional and clean
 - Hosting via GitHub Pages will work without rendering issues
 - You can link between files and pages easily

[→ Back to Table of Contents](#)

Meta-Prompting

"Act as the best prompt engineer. Based solely on the information I provide, craft the most effective and optimized prompt possible, tailored specifically to my goal, audience, and the intended platform: [Platform]. Do not include code, explanations, or external content—only return the final, refined prompt I can use directly."

You can now swap [Platform] with anything—like "ChatGPT", "Midjourney", "Claude", "Notion AI", etc.

I used meta prompting to develop a playable Sudoku game.

The Example process is documented here:  [Meta-prompting](#)

You can play the final game here:  [Sudoku Game](#)

A handy collection of AI prompting cheat sheets I put together to help improve prompt crafting: [GenAI+ Cheat Sheets](#)

A categorized list of AI tools with brief features.

AI Coding & Assistant Platforms (Advanced reasoning, code generation, dev tools, web-based)

- GitHub Copilot – Code completion, IDE integration, real-time suggestions
- ChatGPT – General AI assistant, coding help, online chat interface
- Claude – Long-context reasoning, chat-based AI, strong document handling
- Gemini – Google's AI suite, coding & writing, deeply web-integrated
- Mistral – Open-source models, fast local deployment, modular LLMs

Lightweight AI Builders (Prompt chaining, mini-app creation, lightweight AI systems)

- Lovable – Emotional UX, playful chat experiences, minimal design
- Bolt – Modular prompt workflows, Notion-compatible, logic templates
- Rocket – Quick prompt creation, deploy to web, templates for interactive use
- Vo (vo.dev) – AI-powered UI builder from text, deploy via Vercel

Command-Line & Interface Tools (Terminal tools, AI-enhanced UI, dev productivity boosters)

- Windsurf – AI-enhanced IDE, autocomplete, inline code insight
- Trae – CLI-first tool for managing prompts and AI logic
- Warp – Next-gen terminal with AI command search & suggestions
- Cursor – AI-native code editor, autocomplete, debugging
- Raycast AI – Spotlight-like productivity commands + AI assistant
- CommandBar AI – AI command palettes for SaaS apps, in-app UX layer

Github Desktop - Account and Repo Setup

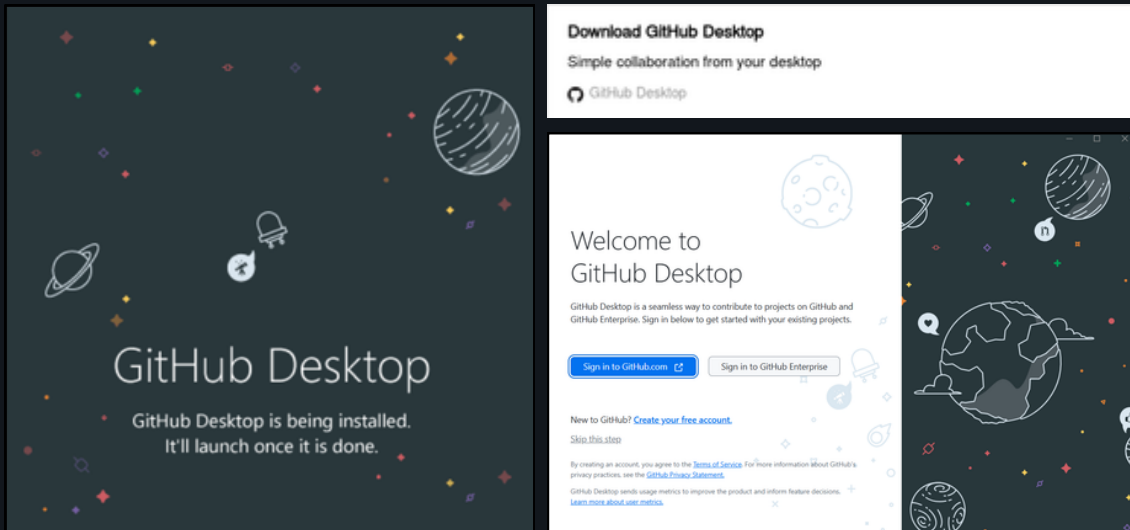
1. Download GitHub Desktop

Go to the official GitHub Desktop website: <https://desktop.github.com/download/>

Click Download for Windows.

Once the download is complete:

- Open the downloaded **.exe** file. → Follow the installation wizard. → Wait for the installation to finish. → GitHub Desktop will launch automatically.



2. Sign in to GitHub

When GitHub Desktop opens for the first time, you'll see the option:

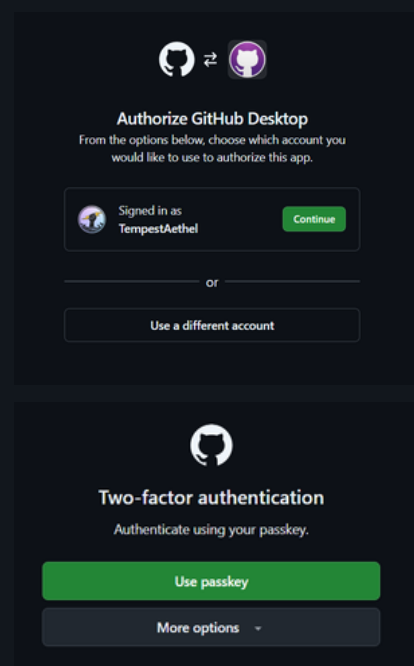
Sign in to GitHub.com → Click it.

If you don't already have a GitHub account, you can choose Create your free account, which will redirect you to GitHub where you can register.

If you already have an account:

- Your browser will open.
- Sign in to your GitHub account.
- If you're already logged in on your browser, simply click Authorize GitHub Desktop.
- If you want to use a different account, sign out in the browser first or log in with another account before authorizing.

Two-Factor Authentication (2FA): If your account has



[→ Back to Table of Contents](#)

2FA enabled, GitHub will ask you to complete the additional authentication step before authorization. Once authorization is complete, GitHub Desktop automatically connects to your account.

3. Configure Git

After signing in, GitHub Desktop lets you configure Git.

Here you choose:

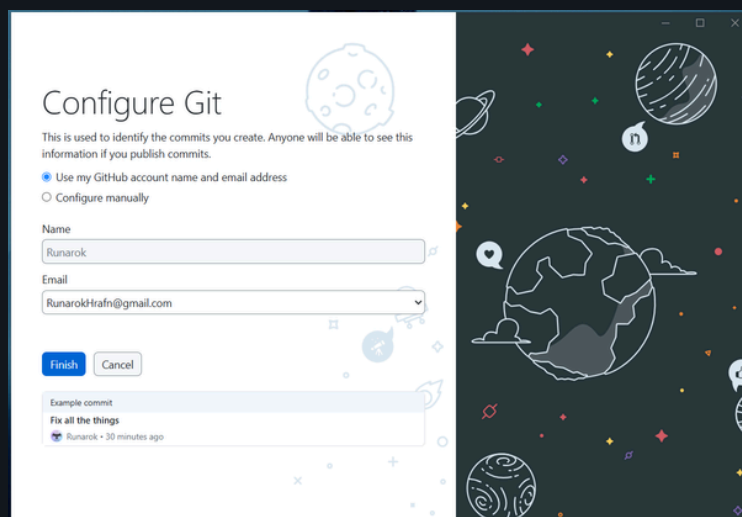
- Your display name (will be same as Username), Your email address

This information is attached to every commit you make.

If you have multiple email addresses linked to your account, choose the email you want your commits to be associated with.

After confirming your settings, click Finish.

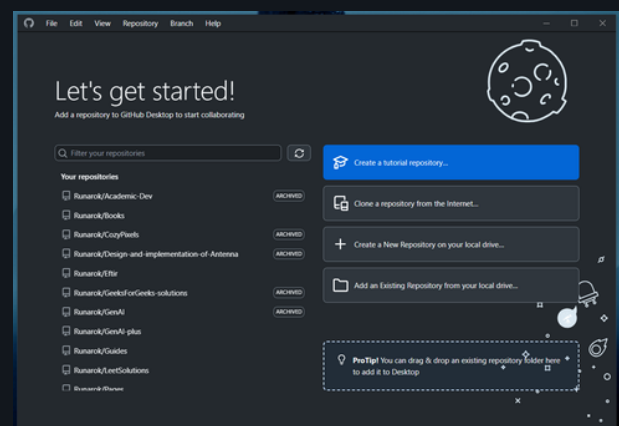
You're now ready to start using GitHub Desktop.



4. Home Screen

For a brand-new account, this page will usually be empty because you don't have any repositories yet.

If you've already used GitHub before, your repositories may already appear here. You'll typically see four main options.



[→ Back to Table of Contents](#)

Option 1 — Create a New Repository on your Local drive

This creates:

- A new project folder on your computer ; A new Git repository inside it.

Use this when you're starting a brand-new project.

You'll choose:

- Repository Name
- Description (optional)
- Local folder location
- Whether to initialize with a README

After creation, you can publish it to GitHub whenever you're ready.

Repository Naming Tip

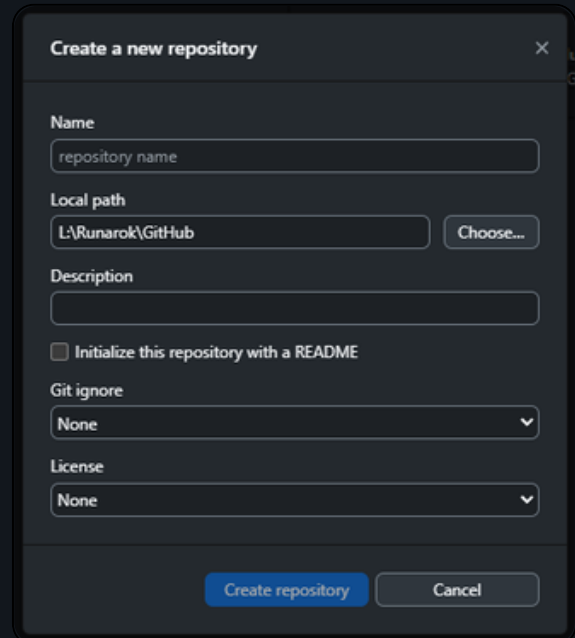
Avoid naming a normal repository the same as your GitHub username unless you intend to use GitHub's special repository features.

For example: If your username is "runarok"

- [runarok](#) – The README.md in this repository is displayed on your GitHub profile (<https://github.com/runarok>), making it your Profile README.
- [runarok.github.io](#) – This repository hosts your main GitHub Pages site at <https://runarok.github.io/>. Unlike normal repositories, it is served from the root URL and does not create a project subpath.

For regular projects, use descriptive repository names such as Portfolio, Calculator, Notes-App, AI-Assistant, or PersonalWebsite. These are published under a subpath, for example:

- [Portfolio](#) → <https://runarok.github.io/Portfolio/>
- [Calculator](#) → <https://runarok.github.io/Calculator/>



Option 2 — Clone a Repository from the Internet

Cloning a repository creates a complete local copy of a Git repository on your computer while keeping it connected to its remote repository (such as GitHub).

Cloning allows you to work on a project without relying on GitHub's web editor. It is ideal when you want to:

- Make multiple or large changes efficiently.
- Use your preferred code editor or IDE.
- Build, run, and test your project locally.
- Work offline and sync changes later.
- Track changes with Git and keep the local and online repositories synchronized.

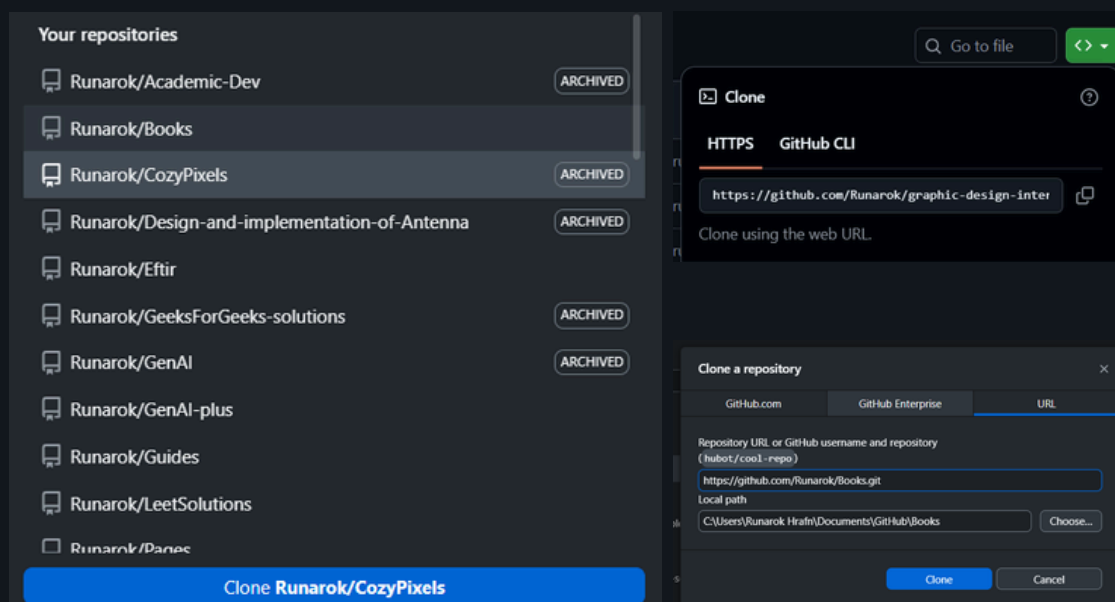
Clone Your Own Repository

- Clone your repository when you want to work on it locally from your computer.
- Your local copy stays connected to the GitHub repository, allowing you to pull updates and push your changes.

Clone Someone Else's Repository

- Clone another user's public repository to explore code, learn, or contribute to the project.
- For private repositories, you need permission and authentication

Note: Public repositories can be cloned by anyone, while private repositories can only be cloned by authorized users (who have SSH key or access token) to clone..



[→ Back to Table of Contents](#)

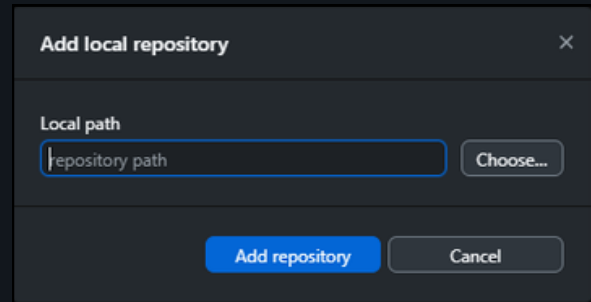
Option 3 — Add an Existing Repository from Local Files

Choose this when:

- You already have a project folder.*
- It is already a Git repository (contains a .git folder).*
- You want to add it to GitHub Desktop so you can manage it there.*

GitHub Desktop does not create anything new, it only adds the existing repository to its list.

*Select your project folder, and
GitHub Desktop will automatically detect the
repository and set it up for you.*

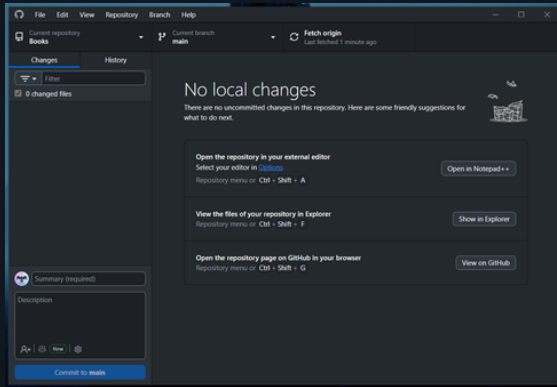


Github Desktop - Basic Repo Commits

Suppose you have just created your first repository on GitHub and want to start working on it locally.

At this point, your local repository and the GitHub repository are exactly the same, so GitHub Desktop shows: **0 Changed Files**

- This means there are no differences between your computer and GitHub.



Now you can open the project in your code editor, add files, modify existing files, and make changes locally. GitHub Desktop will detect these changes and show them under **Changes**, where you can commit and push them back to GitHub.

Editing Files

Open the project folder in your preferred code editor (e.g., VS Code, Visual Studio, Notepad++, or any other editor). Modify the project as needed by editing existing files, adding new files, deleting unnecessary files, renaming files or folders, and saving your changes.

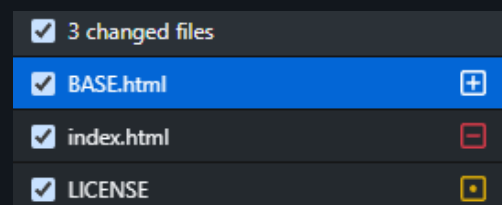
Viewing Changes

Return to GitHub Desktop.

GitHub Desktop automatically detects every change.

You'll see:

- **Added files** – new files created.
- **Deleted files** – files removed.
- **Modified files** – files that were edited.

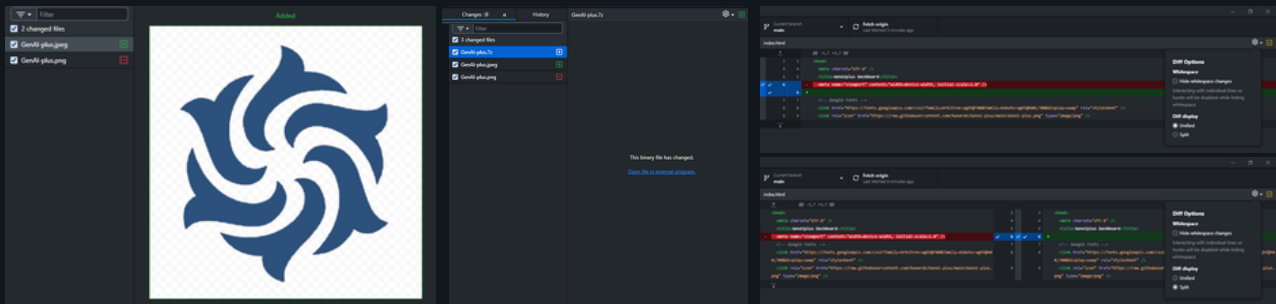


Selecting a file shows exactly what changed.

- GitHub Desktop can display detailed differences for text-based files such as: .md (Markdown), .txt (Text files), .html, .css, .js, images, Source code files

For binary files such as PDFs, videos, or other non-text files, GitHub Desktop can detect that the file changed, but it cannot show a line-by-line preview of what changed inside the file.

[→ Back to Table of Contents](#)



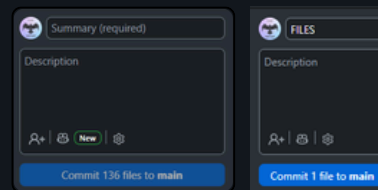
Committing Changes

A commit is like taking a snapshot of your project's current state.

Before committing, enter:

- **Summary (Required):** A short description of what changed
 - Examples: Fixed login bug, Added homepage, Updated README, Added calculator feature
- **Description (Optional):** Provide more detailed information if needed.
 - Example: Added dark mode support, Improved button animations, Fixed mobile layout issues.

When ready: Click **"Commit to main"**
(or whichever branch you're currently using).

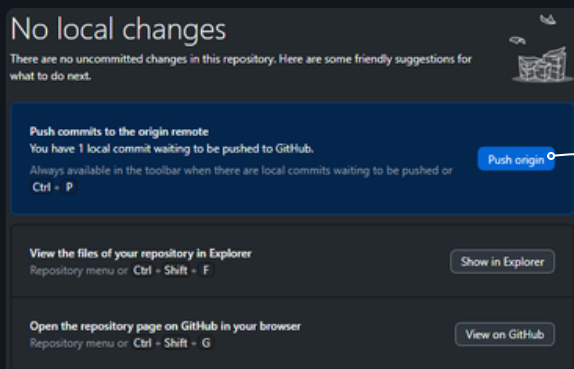
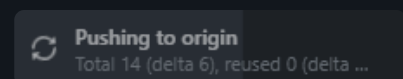


Publishing or Pushing

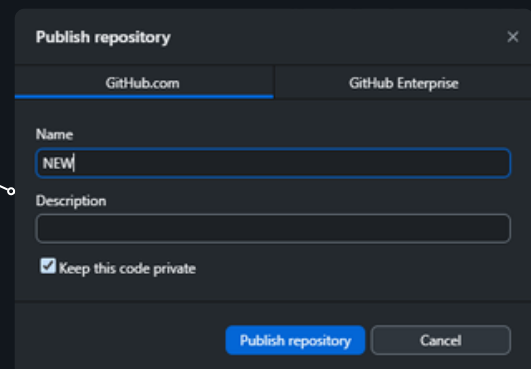
After committing, your changes exist only on your computer.

To upload them to GitHub:

Click **Push Origin**: This sends your commits to the online repository.



Instead, you'll see
Publish Repository. Click to
get...



First-Time Repository

If this repository has never been uploaded to GitHub before, GitHub Desktop won't show Push Origin. Instead, you'll see: **Publish Repository**

[→ Back to Table of Contents](#)

Click it and Choose:

- Repository name
- Description (optional)
- Whether to keep it Public or Private
- Organization (optional)

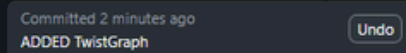
Then click Publish Repository.

GitHub Desktop creates the repository online and uploads your project.

After that, future commits use **Push Origin**.

Undoing a Commit

If you accidentally commit the wrong changes before pushing, GitHub Desktop makes it easy to undo.



Simply click: **Undo** - located in the bottom-left corner of the window.

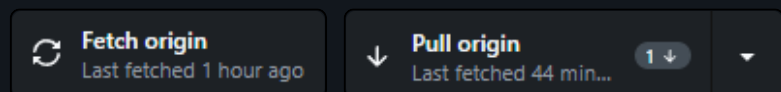
This removes the last commit while keeping your file changes, allowing you to edit them further or create a new commit.

Note: Once you've pushed the commit to GitHub, undoing it is more advanced and should be done with care, especially if others are working on the same repository.

Pulling Changes

If changes have been made to the repository on GitHub or from another computer, you should first check for updates.

Simply click **Fetch Origin**.



GitHub Desktop checks whether any new commits are available in the remote repository.

If updates are found, Fetch Origin changes to **Pull Origin**.

Click **Pull Origin** to download and merge the latest changes into your local repository.

Note: It's a good practice to fetch and pull the latest changes before starting new work to avoid merge conflicts.

Basic Workflow Summary (Beginner)

1. Create or clone a repository.
2. Fetch & Pull (if needed) to get the latest changes from GitHub.
3. Edit your files locally.
4. GitHub Desktop detects the changes.
5. Commit your changes with a summary (and optional description).
6. Push the commit to GitHub (or Publish Repository for a new repository).

The common workflow is:

Fetch & Pull (if needed) → Edit → Commit → Push

You can also make multiple edits and commits locally, then push them together:

Edit → Commit → Edit → Commit.. → Push

This covers the beginner workflow for most projects. As you work on larger projects, GitHub Desktop also supports more advanced version control tasks such as managing branches, switching versions, merging changes, and working with different development workflows.